# Semantic tagging

AI Integration — Cloud LLM APIs · built Nov 13, 2026

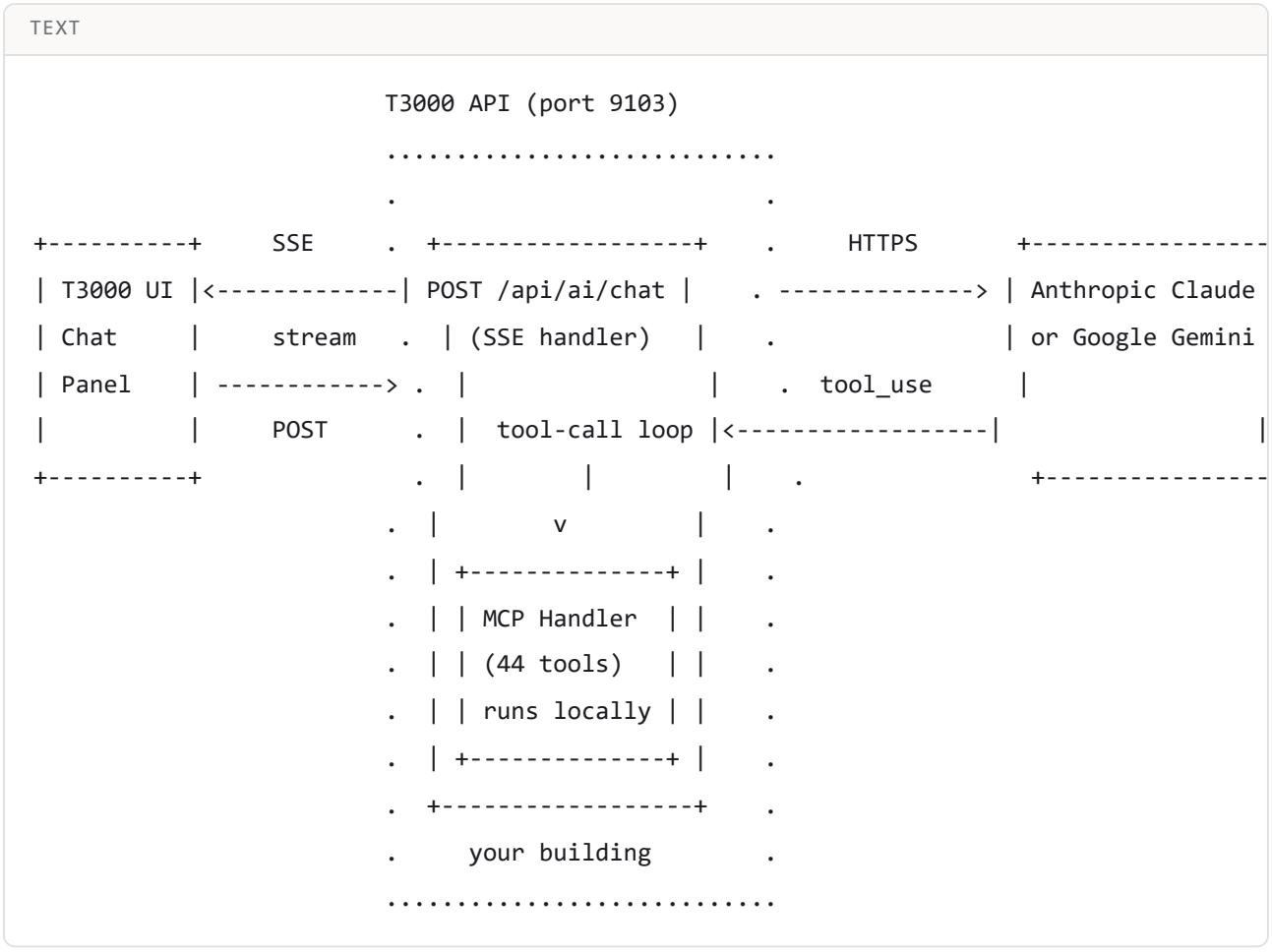
# AI Integration — Cloud LLM APIs

⬅️ Back to Docs | Local LLM Server | AI Chat Page Design

How T3000 connects to cloud-hosted LLM providers (Anthropic Claude, Google Gemini) for the built-in AI chat panel. The T3000 backend acts as a secure proxy — your API key never leaves the server.

## Architecture

TEXT




1. Browser sends user message to `POST /api/ai/chat`
2. T3000 forwards to the cloud LLM (Claude or Gemini) over HTTPS
3. LLM streams response — text chunks go straight to browser via SSE

4. When LLM wants to call a tool, T3000 intercepts the `tool_use` event
5. Tool is executed **locally** by the MCP handler — **no building data sent to the cloud provider**
6. Tool result is sent back to the LLM, which continues the stream

The browser talks only to T3000 via SSE. T3000 forwards to the LLM, intercepts tool calls, executes them against the MCP handler locally, and streams the text response back.

---

## Provider 1: Anthropic (Claude)

### API Endpoint

```
TEXT

POST https://api.anthropic.com/v1/messages
```

### Headers

| Header | Value | Required |
|---|---|---|
| `x-api-key` | `sk-ant-api03-...` | Yes |
| `anthropic-version` | `2023-06-01` | Yes |
| `Content-Type` | `application/json` | Yes |

### Request (with streaming & tools)

JSON

```
{
  "model": "claude-3-5-sonnet-20241022",
  "max_tokens": 4096,
  "system": "You are a building automation assistant for the T3000 platform...",
  "messages": [
    {"role": "user", "content": "What's AHU-1 supply temperature?"}
  ],
  "tools": [
    {
      "name": "t3000_haystack_search_points",
      "description": "Search points by Haystack tags.",
      "input_schema": {
        "type": "object",
        "properties": {
          "tags": {"type": "array", "items": {"type": "string"}, "description": "Tags to
          "serial_numbers": {"type": "array", "items": {"type": "integer"}, "description
        },
        "required": ["tags"]
      }
    },
    {
      "name": "t3000_point_read",
      "description": "Read live value of a single point.",
      "input_schema": {
        "type": "object",
        "properties": {
          "serial_number": {"type": "integer"},
          "point_type": {"type": "string", "enum": ["INPUT", "OUTPUT", "VARIABLE"]},
          "point_index": {"type": "integer"}
        },
        "required": ["serial_number", "point_type", "point_index"]
      }
    }
  ],
  "stream": true
}
```

### Streaming Response (SSE events)

The response is a series of Server-Sent Events:

| Event | Meaning |
|---|---|
| `message_start` | Conversation begins, includes `message.id` |
| `content_block_start` | A new block starts — `text` or `tool_use` |
| `content_block_delta` | Incremental text chunk ( `text_delta` ) or tool args ( `input_json_delta` ) |
| `content_block_stop` | Current block completed |
| `message_delta` | Stop reason ( `end_turn` or `tool_use` ) |
| `message_stop` | Full turn complete |

### Tool Call Cycle

When Claude decides to use a tool, you'll see:

```TEXT
event: content_block_start
data: {"type":"content_block_start","index":0,"content_block":{"type":"tool_use","id":"t

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"input_json_delta","partia
```

After executing the tool locally, send back:

```JSON
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01Axyz...",
      "content": "[{\"label\":\"AHU-1 Supply Temp\",\"value\":55.2,\"units\":\"Deg.C\",\
    }
  ]
}
```

Then continue the conversation — Claude will use the result in its next response.

### Recommended Models

| Model | Use case | Cost |
|---|---|---|
| `claude-3-5-sonnet-20241022` | General queries, dashboard Q&A | $$ |
| `claude-3-haiku-20240307` | Fast lookups, status checks | $ |
| `claude-3-opus-20240229` | Complex diagnostics, multi-AHU analysis | $$$ |

### Get an API Key

1. Go to console.anthropic.com
2. Create an account or sign in
3. Navigate to **API Keys** → **Create Key**
4. Copy the key (starts with `sk-ant-api03-`)
5. Paste into T3000 → AI Settings

---

## Provider 2: Google Gemini

### API Endpoint

```
TEXT

POST https://generativelanguage.googleapis.com/v1beta/models/{MODEL}:streamGenerateConte
```

The API key goes in the URL as a query parameter — not a header.

### Headers

| Header | Value |
|---|---|
| `Content-Type` | `application/json` |

### Request (with streaming & function calling)

JSON

```
{
  "system_instruction": {
    "parts": [{"text": "You are a building automation assistant for the T3000 platform..
  },
  "contents": [
    {
      "role": "user",
      "parts": [{"text": "What's AHU-1 supply temperature?"}]
    }
  ],
  "tools": [
    {
      "function_declarations": [
        {
          "name": "t3000_haystack_search_points",
          "description": "Search points by Haystack tags.",
          "parameters": {
            "type": "object",
            "properties": {
              "tags": {"type": "array", "items": {"type": "string"}},
              "serial_numbers": {"type": "array", "items": {"type": "integer"}}
            },
            "required": ["tags"]
          }
        },
        {
          "name": "t3000_point_read",
          "description": "Read live value of a single point.",
          "parameters": {
            "type": "object",
            "properties": {
              "serial_number": {"type": "integer"},
              "point_type": {"type": "string", "enum": ["INPUT", "OUTPUT", "VARIABLE"]},
              "point_index": {"type": "integer"}
            },
            "required": ["serial_number", "point_type", "point_index"]
          }
        }
```

```
      ]
    }
  ]
}
```

### Streaming Response

Each SSE data line contains a chunk like:

JSON

```
{
  "candidates": [
    {
      "content": {
        "role": "model",
        "parts": [{"text": "The supply "}]
      }
    }
  ]
}
```

When Gemini calls a function:

JSON

```
{
  "candidates": [
    {
      "content": {
        "role": "model",
        "parts": [
          {
            "functionCall": {
              "name": "t3000_haystack_search_points",
              "args": {"tags": ["supply", "air", "temp"]}
            }
          }
        ]
      }
    }
  ]
}
```

### Tool Response Format

Send the result back as a follow-up content:

JSON

```
{
  "contents": [
    {"role": "user", "parts": [{"text": "What's AHU-1 supply temperature?"}]},
    {
      "role": "model",
      "parts": [{"functionCall": {"name": "t3000_haystack_search_points", "args": {"tags
    },
    {
      "role": "function",
      "parts": [
        {
          "functionResponse": {
            "name": "t3000_haystack_search_points",
            "response": {"points": [{"label": "AHU-1 Supply Temp", "value": 55.2}]}
          }
        }
      ]
    }
  ]
}
```

### Recommended Models

| Model | Use case |
|---|---|
| `gemini-2.0-flash` | Fast, great for structured data extraction |
| `gemini-1.5-pro` | Complex reasoning, multi-step tool chains |

### Get an API Key

1. Go to aistudio.google.com/apikey
2. Click **Create API Key**
3. Copy the key
4. Paste into T3000 → AI Settings

---

## Tool Format Mapping

The 25 MCP tools must be converted to each provider's format:

| MCP Tool Schema | Anthropic | Gemini | OpenAI/Local |
|---|---|---|---|
| `input_schema` | `input_schema` (same) | `parameters` (same) | `function.parameters` |
| Array container | Direct `tools[]` | `tools[].function_declarations[]` | `tools[].function` |
| `$ref` / `definitions` | Supported | Flatten manually | Supported |

T3000's backend normalizes all three formats from a single canonical MCP tool definition stored in code.

---

## Rate Limits & Costs

| Provider | Free Tier | Pay-as-you-go |
|---|---|---|
| **Anthropic** | None | $3/M input tokens, $15/M output (Sonnet) |
| **Gemini** | 1,500 requests/day (Flash) | $0.075/M input, $0.30/M output (Flash) |

Tool calls count as output tokens (Anthropic) or are free (Gemini, separate billing).

---

## Security Notes

- T3000 stores API keys encrypted at rest using AES-256-GCM
- The browser never sees the raw API key — it is injected server-side
- All LLM traffic goes through T3000's own HTTPS connection
- Tool execution happens locally — no building data leaves your network to the LLM provider